Nhill 2.1 — Guide de compilation et de livraison

English: ../en/build-guide.md

Comment produire, vérifier et assembler les livrables de la version 2.1.0 sur une machine **neuve**. Tous les scripts sont dans `scripts/`, écrits en Bash strict (`set -Eeuo pipefail`), et partagent `scripts/lib/common.sh`.

1. Ce qui se construit où

Livrable	Machine de construction	Script	Sortie
Serveur + 3 applications (Linux), tests	Debian 13 ou Ubuntu 24.04 amd64	`build-linux.sh`	`build/`
Paquets `.deb`	la distribution **cible** (un jeu par distribution)	`build-deb.sh`	`dist/linux/<distribution>/`
Installateurs Windows (`.exe` NSIS)	Windows 10/11 + Visual Studio 2022 + Qt 6.5 MSVC (recommandé) ; ou Linux + MinGW + Qt pour Windows	`packaging/windows/build-windows.ps1` ; `build-windows.sh`	`build-windows/dist/` ; `dist/windows/`
APK Android (Nhill Business)	Linux + JDK 17 + SDK/NDK Android + Qt pour Android	`build-android.sh`	`dist/android/`
PDF de la documentation	toute machine avec pandoc + moteur PDF	`build-docs-pdf.sh`	`docs/pdf/`, `legal/pdf/`, `marketing/pdf/`, `README*.pdf`
Site vitrine (archive)	toute machine avec Python 3	`build-website.sh`	`dist/website/`
Contrôle des livrables	—	`verify-build.sh`	`build/logs/verify-build-*.log`
Dossier de livraison	—	`package-release.sh`	`release/V2.1.0/`
Tout ce qui est possible ici	—	`build-all.sh`	résumé réussi / ignoré / échoué

Un paquet `.deb` dépend des versions de bibliothèques de sa distribution : ne copiez pas un paquet Ubuntu sur Debian (ni l'inverse). La CI construit les deux jeux dans des conteneurs propres.

2. Conventions communes

Option / code	Signification
`--dry-run`	affiche les commandes sans rien exécuter ni installer
`--force`	autorise l'écrasement d'un dossier de sortie existant
`--output DIR`	dossier de sortie
`--server-url URL`	**retirée en 3.0.0** : refusée ; aucune adresse de serveur n'est compilée, l'utilisateur la saisit au premier lancement (vérifiée, puis enregistrée sur l'appareil)

Option / code	Signification
`-h`, `--help`	aide
code 0	succès
code 1	échec (compilation, test, contrôle)
code 2	mauvaise utilisation (option inconnue)
code 3	environnement incompatible (système, outil ou SDK absent) — le message indique quoi installer

Journaux : `build/logs/<script>-<date>.log`. Chaque dossier de sortie reçoit un fichier `SHA256SUMS`. Aucun script ne contient de secret ; les clés de signature se donnent par variables d'environnement au moment de la construction.

3. Linux

3.1 Dépendances (machine neuve)

```
scripts/install-deps-linux.sh --dry-run --all      # voir ce qui serait installé
scripts/install-deps-linux.sh --with-packaging --with-docs --yes
```

Uniquement les dépôts officiels de la distribution (`apt`). Distributions validées : Debian 13, Ubuntu 24.04 (`--force-distro` pour essayer une dérivée, sans garantie). Un inventaire des versions installées est écrit dans `build/logs/deps-manifest-*.txt`.

Sans accès aux dépôts `apt` (réseau filtré) : `scripts/bootstrap-deps-from-source.sh` compile les dépendances depuis des étiquettes précises de leurs dépôts officiels dans un préfixe privé. Méthode de secours : préférez les paquets de la distribution, qui reçoivent les correctifs de sécurité (cpp-httplib 0.14 en particulier, voir l'audit de sécurité).

3.2 Compilation et tests

```
scripts/build-linux.sh                          # tout + tests
scripts/build-linux.sh --server-only --jobs 4
scripts/build-linux.sh --client-only --no-tests
```

3.3 Paquets Debian

```
scripts/build-deb.sh
# -> dist/linux/ubuntu-24.04/nhill-{server,business,pos,console}_3.0.0-1_amd64.deb
#    (ou dist/linux/debian-13/ sur Debian 13)
```

Les dépendances de chaque paquet sont calculées par `dpkg-shlibdeps` ; `lintian` contrôle les paquets. Contrôle d'installation sur une machine propre : `sudo apt install ./dist/linux/<distribution>/nhill-business_3.0.0-1_amd64.deb`.

4. Windows

4.1 Sur Windows (méthode recommandée)

Prérequis : Visual Studio 2022 (charge « Développement C++ »), CMake ≥ 3.21, Ninja, Git, Qt 6.5.3 MSVC 2019 64 bits, NSIS 3. QtKeychain (« Se souvenir de moi », Gestionnaire d'identification Windows) n'a pas à être installé : le script le compile depuis sa source officielle (étiquette 0.17.0, commit vérifié) avec `packaging\windows\build-qtkeychain.ps1`, en bibliothèque **statique**.

```
powershell -ExecutionPolicy Bypass -File packaging\windows\build-windows.ps1 `
  -Source C:\src\nhill -QtDir C:\Qt\6.5.3\msvc2019_64 `
  -BuildDir C:\src\nhill\build-windows
```

`-ExecutionPolicy Bypass` ne s'applique qu'à cette commande ; ne modifiez pas la stratégie d'exécution de la machine. Résultat, dans `build-windows\dist\` : `nhill-business-2.1.0-setup.exe`, `nhill-pos-2.1.0-setup.exe`, `nhill-console-2.1.0-setup.exe` et `SHA256SUMS`.

QtKeychain sous Windows :

Option	Effet
(aucune)	QtKeychain est compilé dans `<BuildDir>\deps\qtkeychain` à la première construction, puis réutilisé
`-KeychainDir C:\build\qtkeychain`	utilise une construction statique existante (produite par `build-qtkeychain.ps1 -QtDir … -Prefix …`)
`-AllowNoKeychain`	construit sans QtKeychain : la session n'est **jamais** mémorisée (aucun repli en clair)

Pourquoi une bibliothèque statique : `windeployqt` traite toute DLL nommée `Qt6*.dll` comme un module Qt et la cherche uniquement dans le dossier de Qt ; une `qt6keychain.dll` livrée à côté de l'application fait donc échouer le déploiement. Liée statiquement, QtKeychain est incluse dans chaque `.exe`. Après la compilation, le script vérifie avec `dumpbin` que chaque application importe `CredReadW`, `CredWriteW` et `CredDeleteW` d'`ADVAPI32` et ne dépend d'aucune DLL QtKeychain. Le paquet vcpkg de QtKeychain (une DLL) n'est donc pas utilisé, même avec `-VcpkgRoot`.

Test réel du stockage sécurisé : `-DNHILL_BUILD_KEYCHAIN_TEST=ON` (activé par le script) produit `build-windows\tests\keychain\test_keychain.exe` ; déployé avec `windeployqt`, il écrit une session factice dans le Gestionnaire d'identification, la relit depuis un second processus, puis l'efface (`NHILL_KEYCHAIN_REQUIRED=1` transforme l'absence de stockage en échec au lieu d'un test ignoré).

4.2 Depuis Linux (compilation croisée)

```
scripts/install-deps-linux.sh --with-windows-cross --yes
packaging/windows/build-qt-mingw.sh              # Qt pour Windows (plusieurs heures)
scripts/build-windows.sh --qt-prefix /opt/nhill-win/qt6
```

Limite : la compilation croisée MinGW n'intègre pas QtKeychain ; les applications ainsi produites ne mémorisent jamais la session (elles restent sûres, l'option est simplement indisponible). Pour livrer « Se souvenir de moi » sous Windows, utilisez la méthode 4.1.

4.3 Signature

Signez les `.exe` et les installateurs (Authenticode, `signtool sign /fd sha256 /tr <horodatage> …`) avec le certificat de l'éditeur avant toute distribution. Sans signature, SmartScreen avertit l'utilisateur (voir le guide d'installation).

5. Android

```
QT_ANDROID=~/Qt/6.5.3/android_arm64_v8a QT_HOST=~/Qt/6.5.3/gcc_64 \
ANDROID_SDK_ROOT=~/Android/Sdk ANDROID_NDK_ROOT=~/Android/Sdk/ndk/25.2.9519653 \
scripts/build-android.sh
# -> dist/android/nhill-business-3.0.0-arm64-v8a.apk
```

QtKeychain (« Se souvenir de moi », Keystore Android) est obligatoire. Construisez-le une fois depuis sa source vérifiée (étiquette 0.17.0, bibliothèque statique), puis indiquez son préfixe :

```
QT_ANDROID=~/Qt/6.5.3/android_arm64_v8a packaging/android/build-qtkeychain-android.sh
~/build/qtkeychain-android
KEYCHAIN_PREFIX=~/build/qtkeychain-android QT_ANDROID=… QT_HOST=… ANDROID_SDK_ROOT=…
ANDROID_NDK_ROOT=… \
scripts/build-android.sh
```

Sans `KEYCHAIN_PREFIX`, la construction s'arrête ; `ALLOW_NO_KEYCHAIN=1` produit un APK qui ne mémorise jamais la session. Le script vérifie que le code Keystore de QtKeychain (`KeyGenParameterSpec`) figure dans la bibliothèque de l'application. Le test réel (`.github/scripts/android-keychain-emulator.sh`) installe `test_keychain` sur un émulateur Android 11 x86_64 et vérifie l'écriture, la relecture après redémarrage de l'application, la déconnexion et l'effacement.

Le SDK, le NDK et Qt pour Android ne viennent pas d'`apt` : installez-les depuis leurs sources officielles (sdkmanager de Google, installateur en ligne de Qt) et vérifiez les empreintes publiées. Signature de publication : `KEYSTORE=… KEYSTORE_ALIAS=… KEYSTORE_PASS=…` ; sans ces variables, l'APK est signé avec une clé de **débogage** (installable par `adb`, non publiable). Le keystore ne doit jamais être versionné ; en CI il est fourni par les secrets `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_ALIAS`, `ANDROID_KEYSTORE_PASS`.

6. Documentation PDF

```
scripts/build-docs-pdf.sh            # régénère tous les PDF listés dans docs/pdf-manifest.txt
scripts/build-docs-pdf.sh --check    # échoue si un PDF manque ou ne correspond plus à son
Markdown
```

Le Markdown est la seule source ; `docs/pdf/manifest.json` enregistre l'empreinte de chaque source et de chaque PDF. Ne modifiez jamais un PDF à la main.

7. Construire avec GitHub Actions

Les plateformes impossibles à construire sur votre machine se construisent dans la CI :

1. Poussez le dépôt sur un dépôt GitHub **privé**.
2. Déclenchez les workflows : étiquette `git tag v2.1.0 && git push origin v2.1.0`, ou « Run workflow » (onglet Actions) pour `linux-packages`, `windows`, `android`, `docs-website`.

3. Téléchargez les artefacts : `nhill-deb-ubuntu-24.04`, `nhill-deb-debian-13`, `nhill-windows-installers`, `nhill-android-apk`, `nhill-docs-and-website`.
4. Placez-les dans `dist/linux/ubuntu-24.04/`, `dist/linux/debian-13/`, `dist/windows/`, `dist/android/`, `dist/website/`, puis § 8.

Avec l'outil `gh` : `gh run download <id> -n nhill-windows-installers -D dist/windows`.

8. Vérifier et livrer

```
scripts/verify-build.sh --require linux,docs        # contrôle de ce qui existe
scripts/package-release.sh --force                  # assemble release/V2.1.0/
```

`verify-build.sh` vérifie, pour chaque plateforme présente : le format réel des fichiers (paquet Debian, exécutable PE, APK), la version 2.x, les sommes `SHA256SUMS`, la présence et la fraîcheur des PDF. `package-release.sh` copie uniquement ce qui existe dans `release/V2.1.0/{linux,windows,android,website,documentation,legal,marketing,source,checksums}` et liste ce qui manque dans `checksums/MISSING.txt` — jamais de fichier factice.

9. Construction reproductible : points d'attention

- Construisez à partir d'un commit étiqueté et propre (`git status` vide) ; l'archive des sources est produite par `git archive`.
- Notez les versions des outils (`deps-manifest-*.txt`, journaux de la CI).
- Les dates intégrées aux paquets dépendent de l'horloge de construction ; deux constructions ne sont pas forcément identiques octet pour octet.